LÖSUNGEN · LEHREREXEMPLAR

W1·1 – Klassen und Objekte: Lösungen zu den Papier-Aufgaben

Zum Arbeitsblatt `AQ_W1_AB1_Klassen_und_Objekte.docx` · IDE-Level: siehe `AQ_W1_L1_Heldenschmiede_LSG.txt`

Aufgabe 0 – Heldenkarten

b) Gemeinsam: dieselben Attribute (Klasse = Bauplan). Verschieden: die Attributwerte (jedes Objekt hat seinen eigenen Zustand). **d)** Klassenkarte Monster wie Abbildung 2 auf dem AB (3 private Attribute, Konstruktor mit 3 Parametern, Getter, nimmSchaden, istBesiegt, angreifen).

Aufgabe 2 – Klasse erläutern (8 BE)

a) Mustertext: Die Klasse Monster beschreibt die Gegner im Spiel. Jedes Monster-Objekt hat drei Attribute: name (Zeichenkette), leben und staerke (Ganzzahl). Das Zeichen – kennzeichnet sie als privat; sie sind nur innerhalb der Klasse direkt zugreifbar. Der Konstruktor (c) Monster(name, leben, staerke) erzeugt ein neues Monster und belegt die Attribute mit den übergebenen Startwerten. Die mit + gekennzeichneten öffentlichen Methoden bilden die Schnittstelle nach außen: Drei Getter liefern die Attributwerte. nimmSchaden(schaden) verringert leben (nie unter 0), istBesiegt() liefert wahr, wenn leben ≤ 0 ist. angreifen(h) lässt den übergebenen Helden Schaden in Höhe der eigenen Stärke nehmen. „nichts“ bedeutet: keine Rückgabe.

Kernpunkte: Attribute mit Typen und Sichtbarkeit (1) · Konstruktor mit Aufgabe (1) · Methoden sinnvoll gruppiert, Rückgabetypen (1) · Zusammenhang: Zugriff nur über Methoden (1).

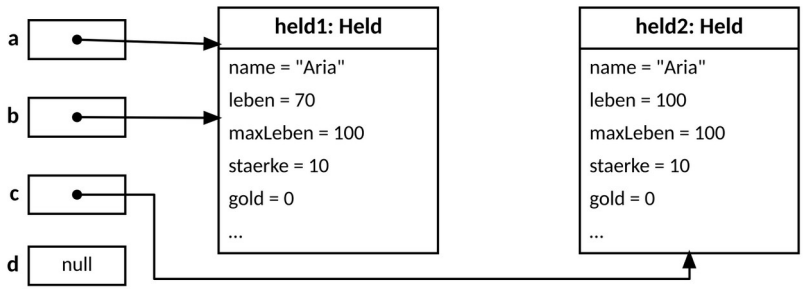
b) Kapselung: Von außen kann leben nur über nimmSchaden verändert werden. Die Methode stellt sicher, dass leben nie negativ wird. Ein öffentliches Attribut könnte jede Klasse auf einen ungültigen Wert setzen (2 BE).

c) Sondierend: getName, getLeben, getStaerke, istBesiegt. Verändernd: nimmSchaden; angreifen verändert den Zustand des übergebenen Helden (2 BE).

Aufgabe 3 – Überladene Konstruktoren

b) `experte_heilenMitEigenemMaxLeben`: heilen muss maxLeben verwenden, nicht die feste Zahl 100. **c)** `new Held("Aria")` → Held(name) · `new Held("Brann", 150, 14)` → Held(name, maxLeben, staerke) · `new Held("Cora", 80)` → Übersetzungsfehler: Es gibt keinen Konstruktor mit den Parametern (Zeichenkette, Ganzzahl).

Aufgabe 4 – Der Doppelgänger



a) Zwei Held-Objekte, vier Variablen. `Held b = a;` kopiert nur die Referenz: a und b verweisen auf dasselbe Objekt (Objektnamen frei wählbar).

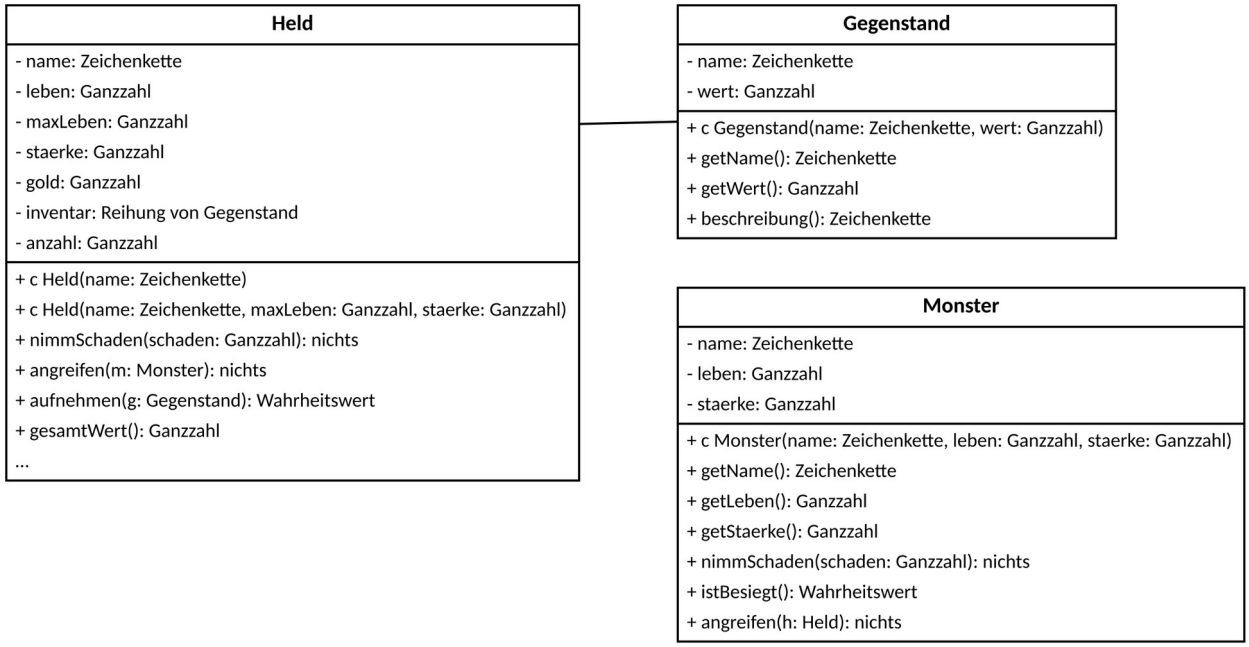
b) `70` (Schaden über b trifft dasselbe Objekt) · `100` (c ist ein eigenes, neues Objekt) · `true` (a und b enthalten dieselbe Referenz) · `false` (gleiche Attributwerte, aber verschiedene Objekte – `==` vergleicht bei Objekten Referenzen).

c) d verweist auf kein Objekt (null). Der Methodenaufruf führt zu einem Laufzeitfehler (NullPointerException), das Programm bricht ab. Abhilfe: vorher `if (d != null)` prüfen.

Aufgabe 5 – Monster und Kampf

d) Runde 1: Goblin 20, Aria 94 · Runde 2: Goblin 10, Aria 88 · Runde 3: Goblin 0 → besiegt, schlägt nicht mehr zurück. **3 Runden, Aria hat 88 Leben. Denkanstoß:** leben ist in Monster privat – `m.leben` ist in Held ein Übersetzungsfehler. Außerdem sorgt nur nimmSchaden für „nie unter 0".

Aufgabe 7 – Die Karte der Schmiede



a) Kernpunkte (5 BE): drei Klassenkarten mit + / − (1) · Attribute mit deutschen Typnamen, inventar als Reihung von Gegenstand (1) · Konstruktoren mit c, beide Held-Konstruktoren (1) · Methoden mit vollständiger Signatur, Rest „…" (1) · Assoziation Held – Gegenstand als einfache Linie ohne Beschriftung und Kardinalität (1).

b) Held hat ein Attribut vom Typ Reihung von Gegenstand – eine dauerhafte Beziehung (Assoziation). Ein Monster-Objekt wird nur als Parameter von angreifen kurz benutzt; Held speichert keine Referenz auf ein Monster. Darum keine Linie (2 BE).

Aufgabe 8 – Der Papier-Compiler: wertvollsterGegenstand()

```
public Gegenstand wertvollsterGegenstand() {
    if (anzahl == 0) {
        return null;
    }
    Gegenstand bester = inventar[0];
    for (int i = 1; i < anzahl; i++) {
        if (inventar[i].getWert() > bester.getWert()) {
            bester = inventar[i];
        }
    }
    return bester;
}
```

Typische Fehler:

- Keine Leer-Abfrage → Zugriff auf inventar[0] liefert null, danach Laufzeitfehler bei getWert().
- Schleife bis `inventar.length` statt `anzahl` → Zugriff auf null-Plätze.
- `>=` statt `>` → bei Gleichstand wird der letzte statt der erste geliefert.

- Index statt Gegenstand zurückgegeben; Rückgabe fehlt in einem Pfad; Attribut `wert` direkt statt `getWert()` (privat in Gegenstand).

Für Schnelle – Die Arena (mögliche Lösung)

```
public class Arena {
    public static boolean kampf(Held h, Monster m) {
        int runde = 0;
        while (!h.istBesiegt() && !m.istBesiegt() && runde < 100) {
            runde++;
            int wurf = Wuerfel.wuerfeln(6);
            if (wurf == 6) {
                h.angreifen(m);
                h.angreifen(m);
            } else if (wurf != 1) {
                h.angreifen(m);
            }
            if (!m.istBesiegt()) {
                m.angreifen(h);
            }
        }
        return m.istBesiegt();
    }
}
```

Neue Objekte je Kampf, weil Held und Monster ihren Zustand (leben) behalten. Simulationswerte (zur Orientierung): Aria gegen Troll gewinnt etwa **17 %** der Kämpfe; für mindestens 90 % braucht sie etwa **maxLeben 170–180** (Zufallsschwankungen).